Microscaling Redundancy and the Post-Training Floor: Quantization-Aware Training Closes the Rest of the NVFP4 Accuracy Gap

Deep Sheth

June 2026

Abstract

NVFP4, the E2M1 grid with a per-16-element FP8 block scale, is the native 4-bit format of Blackwell GPUs. Many methods try to close its accuracy gap to FP8 with rotations, permutations, scaling heuristics, and rank allocation; most contribute little once the format already applies a per-block scale. A high-resolution white-error model shows the output distortion of a linear layer depends only on the per-block maxima and the residual subspace, pointing to two levers: per-channel equalization and additive low-rank correction. After equalization, additive correction gives most of the remaining gain; a rotation adds a small residual benefit, several times smaller. A codec built from these two levers reaches 6.05 perplexity with 4-bit activations on Llama-3.1-8B (FP8: 5.948), and 6.36 with weights, activations, and the KV cache all at 4-bit, under one byte per weight. This post-training codec sits at a floor set by a near-white residual, which motivates training the model directly against the deployment quantizer. Quantization-aware training (QAT), using the exact quantizer that ships, closes 70% of the W4A4 gap on TinyLlama-1.1B and reproduces on real Blackwell hardware with no sim-to-deploy gap (9.709 deployed, FP16 9.358; a 4-bit KV cache costs nothing further, 9.705). QAT, QAT with a 4-bit KV cache, and QAT with added equalization converge to the same floor, 0.32 above FP8, which we argue is fundamental: FP8 is already nearly lossless, and training the weights cannot remove the runtime activation and KV quantization noise that remains. The result is a 4-bit model deployable on its own; we also evaluate it as a speculative-decoding draft (88.9% acceptance against an FP16 target). Measured Blackwell throughput shows NVFP4's speed edge over FP8 only at large batch size, so such methods are best judged on accuracy per byte and memory rather than single-stream speed.

1 Introduction

Inference on Blackwell hardware uses NVFP4, an E2M1 grid with values $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$ and a per-16-element FP8 (E4M3) block scale. The format differs from earlier research formats in that the grid and the group size are fixed by the tensor cores rather than chosen by the method. Existing approaches to the 4-bit accuracy gap apply input transforms such as Hadamard [1] and learned rotations [12] or channel permutations, weight-scaling heuristics such as AWQ [11], error-feedback rounding such as GPTQ [7], and low-rank residual reconstruction such as LQER [14]. Recent work also optimizes the NVFP4 scales themselves [2, 5, 8]. In our experiments, when several of these methods are applied on top of microscaling and after equalization, the marginal improvement is small. This paper offers an explanation under a stated approximation and uses it to construct a compact codec.

This paper makes four main contributions, plus a fifth on where the approach runs out of room. Section 3 gives an approximate distortion model for the microscaling quantizer and a proposition identifying two first-order levers, equalization and additive low-rank correction; several rotation, permutation, scale-objective, and coordinated-rounding variants we tested contribute little beyond these two operations once equalization has been applied. Section 4 builds a codec from these two operations, and Section 5 evaluates it on perplexity, on the ablations the model predicts, on four models, and on the composition of 4-bit weights, activations, and KV cache. The post-training codec sits at a floor set by a residual that is already close to white, which is where Section 6 picks up: training the model directly against the deployment quantizer (QAT) keeps paying off where post-hoc correction cannot, closing 70% of the remaining W4A4 gap on TinyLlama-1.1B, reproducing end to end on real Blackwell hardware with no sim-to-deploy gap, and converging to a characterized all-4-bit floor across three independently motivated methods. The outcome, at each scale tested, is a 4-bit model deployable on its own, which we additionally measure as a speculative-decoding draft. Section 7 reports measured Blackwell throughput and discusses what that implies for how the method should be positioned. Throughout, we try to be clear about which claims rest on the approximation and which experiments are still open.

2 Background and NVFP4 semantics

A block B of $K = 16$ elements is quantized as $\hat{x}_i = s_B\, Q_{\mathrm{E2M1}}(x_i/s_B)$, where s_B is a per-block FP8 (E4M3) scale and Q_{E2M1} rounds to the 16-value grid with maximum $g = 6$. NVFP4 represents values in E2M1 with magnitude up to ± 6, uses one E4M3 scale per block of 16 values, and applies an additional per-tensor FP32 scale [13]; weights may use two-dimensional block scaling while activations use one-dimensional scaling.

To keep the experiments reproducible we state the semantics we use. We take $s_B = \max_{i \in B} |x_i|/g$ rounded to E4M3, one-dimensional per-16 scaling along the contraction axis, an optional per-tensor FP32 scale, and deterministic round-to-nearest on the grid. Weights are quantized symmetrically. For activations we allow a per-block zero-point chosen as the better of zero and the block midpoint; the zero-point folds into a fused epilogue as the cross-term between the shift and the precomputed per-block row sums of the weight, so it adds no per-element metadata and the main matrix multiply stays on the E2M1 grid with an E4M3 scale. We treat this as hardware-compatible rather than deployed hardware-native until the epilogue is implemented in a production kernel. We use numerical (fake) quantization: tensors are rounded to the exact E2M1 grid and the E4M3 scales, and the matrix multiply is then performed in higher precision. This reproduces the numerical values of the stated fake-quantization semantics, but not every production NVFP4 kernel configuration, since we do not model two-dimensional weight scaling or kernel-level rounding modes, and we quantize all linear layers including the language-model head. Accuracy numbers are therefore exact for the semantics stated here.

3 An approximate distortion model

Consider a linear layer $y = Wx$ with $W \in \mathbb{R}^{m \times d}$. Let $e = \hat{x} - x$ be the activation quantization error and $G = W^\top W$. The output distortion is $D = \mathbb{E}[e^\top G\, e]$.

Assumption 1 (High-resolution white error)**.** *Within a block the quantization error is zero-mean and uncorrelated across coordinates, with variance proportional to the squared block scale, so that* $\mathbb{E}[e_i e_j] = \delta_{ij}\, \kappa\, s_B^2$ *for a grid-dependent constant* κ*.*

Assumption 1 is the standard high-resolution model and is approximate at 4 bits, where the grid is coarse. Under it, with $s_B \propto \max_{i \in B} |x_i|$,

$$D \approx \frac{\kappa}{g^2} \sum_b \Big(\max_{i \in B_b} x_i^2 \Big) \operatorname{tr}(G_{B_b B_b}), \tag{1}$$

where $G_{B_b B_b}$ is the diagonal block of G on the coordinates of block b. In Equation (1) a preprocessing method can change only two quantities: the per-block maxima $\max_{i \in B_b} x_i^2$ and, through an additive correction, the residual that survives quantization. The following observations describe how the candidate methods act on these quantities.

Observation 1 (Equalization lowers the block maxima)**.** *A per-channel scaling $x_i \mapsto x_i / s_i$, folded into W, acts before blocking and reduces the factor $\max_{i \in B_b} x_i^2$ in Equation (1). Among multiplicative transforms it is the one that reshapes the block maximum without mixing coordinates.*

Observation 2 (Rotation is approximately neutral near the Gaussian)**.** *A random orthogonal transform applied within a block tends to make the coordinates more Gaussian, though for the small block size $K = 16$ this effect is partial. A Gaussian block is distributionally invariant under further rotation, so its ratio of peak to average magnitude is unchanged; at $K = 16$ we measure it to remain near 4.55. The same transform reduces this ratio for heavy-tailed blocks (from 14.5 to 3.5 in our measurements) and increases it for sub-Gaussian blocks (from 1.2 to 4.5). After equalization the blocks sit closer to the Gaussian regime, which is consistent with the small marginal effect of rotations and permutations we observe in Section 5. This is an empirical regularity under our settings, not a universal law; rotations may well help in other regimes, and learned rotations have been reported to do so [12].*

Observation 3 (Additive low-rank targets the residual)**.** *The minimum-distortion rank-k correction of the residual in the G metric is the top-k Karhunen-Loève transform of the residual covariance $\Sigma_r = W\, \mathbb{E}[ee^\top]\, W^\top$, which is the additive form used by low-rank error reconstruction [14]. No input transform produces this additive term, and a one-bit sign-projection relaxation recovers part of it at low storage cost.*

Proposition 1 (Two first-order levers)**.** *Under Assumption 1 and the microscaling quantizer, the distortion model (1) is reduced through two operations: equalization, which lowers the per-block input maxima (Observation 1), and additive low-rank correction, which reduces the residual subspace (Observation 3). In our post-training NVFP4 setting, once equalization has been applied, additive correction provides substantially more improvement than the remaining multiplicative transforms; an orthogonal transform still contributes a small residual gain, but it is several times smaller than the gain from additive correction (Section 5).*

Proposition 1 describes the approximate model together with an empirical claim about our setting; it is not an optimality theorem. Rotation is not useless after equalization: we measure a small residual benefit that is consistent with reports that learned rotations help [12], though additive correction dominates it. Recent scale-optimization methods report further gains from adjusting block and tensor scales [2, 5, 8], which we do not evaluate here.

4 Method

We implement the two operations. For activations we calibrate per-channel equalization scales on a few hundred WikiText windows and fold them so that the runtime applies x/s and rescales the

output. We add a rank-r correction of the residual computed in the weight-Hessian metric, followed by a per-128 quantized Johnson-Lindenstrauss (QJL) sign-projection correction of the remaining residual with projection dimension 64. For activations we use an SVD side-channel of rank $r = d/16$ per layer, chosen to match the side-channel bit budget of the QJL projection. For weights we apply GPTQ rounding [7] together with an additive low-rank correction of $W - Q(W)$ in the activation-Hessian metric, storing the factors in FP8 with rank $r = d/6$. For the KV cache we apply NVFP4 quantization to the key and value projections. Each correction is folded offline or applied as a rank-r epilogue, so the main matrix multiply remains NVFP4. The compute cost of the epilogue is discussed in Section 7.

5 Experiments

5.1 Accuracy

Table 1 reports perplexity on Llama-3.1-8B over full WikiText-2, with FP16 at 5.918 and FP8 (per-token E4M3 activations) at 5.948.

Table 1: Perplexity on Llama-3.1-8B, full WikiText-2.

Configuration	PPL	Δ vs. FP8
Raw NVFP4 activations	6.263	+0.315
with equalization and zero-point	6.168	+0.220
with SVD side-channel	6.065	+0.117
with QJL	6.050	+0.102
4-bit weights and activations (rank $d/6$)	6.258	+0.310
4-bit weights, activations, and KV cache	6.364	+0.416

5.2 Ablations

Table 2 reports techniques that the distortion model predicts to add little after equalization, with perplexities on Llama-3.1-8B against the relevant baseline.

Table 2: Marginal effect of additional techniques, Llama-3.1-8B.

Technique	Stack	PPL	Baseline
Per-token L2 normalization	A4	6.263	6.263
Per-block Hadamard (fixed mask)	A4	6.052	6.057
Channel permutation	A4	6.051	6.057
Output-weighted scale objective	A4	6.052	6.050
AWQ with GPTQ	W4A4	6.581	6.552
Per-layer rank allocation	W4A4	6.314	6.292

5.3 Equalization, rotation, and additive correction

Table 3 isolates the contributions of rotation and additive correction on Llama-3.1-8B, reporting both perplexity and draft acceptance. Rotation (a tiled Walsh-Hadamard transform applied in the QuaRot manner) improves on raw NVFP4 and gives a further small improvement after equalization,

from 6.155 to 6.136 perplexity. The additive SVD and QJL correction gives a larger improvement over equalization, from 6.155 to 6.050, about five times the marginal effect of rotation, and the largest gain in acceptance. This supports the claim that additive correction is the dominant lever while showing that rotation is not strictly redundant. The values in this table come from a single self-consistent run with acceptance measured on a 4,096-token subset, and differ by up to 0.01 perplexity from Table 1, which aggregates separate runs; both place the full activation codec at 6.050.

Table 3: Activation-quantization levers on Llama-3.1-8B. Acceptance α is measured against the FP16 target over 4,096 tokens.

Configuration	PPL	α
Raw NVFP4	6.259	0.900
with rotation (Hadamard)	6.185	0.911
Equalization	6.155	0.917
with rotation	6.136	0.921
with additive SVD and QJL (ours)	6.050	0.939

5.4 Cross-model evaluation

Table 4 reports the activation codec on four models from two families. For the models of 7B parameters and above the gap to FP8 is between 0.06 and 0.10 perplexity. The largest fraction of the gap closed occurs on Qwen2.5-7B, a model from a different family than the one used for development, which indicates the result is not specific to a single model or family.

Table 4: Activation codec across models, perplexity on WikiText-2. Corrected is the full activation codec; Δ is its gap to FP8; closed is the fraction of the raw-to-FP8 gap recovered.

Model	FP16	FP8	Raw NVFP4	Corrected	Δ vs. FP8	Closed
Llama-3.2-1B	9.283	9.340	10.018	9.699	+0.359	47%
Llama-3.2-3B	7.438	7.466	7.776	7.573	+0.107	65%
Llama-3.1-8B	5.918	5.948	6.263	6.050	+0.102	68%
Qwen2.5-7B	6.512	6.542	6.826	6.598	+0.056	80%

5.5 Byte accounting

Table 5 accounts for the storage of the W4 weights used in the W4A4 configuration; activations are ephemeral and the KV cache is accounted separately. The low-rank factor cost is $r/m + r/d$ bytes per element for an $m \times d$ layer at rank r, and the reported value uses $r = d/6$ with the layer shapes of Llama-3.1-8B. The total stays below the one byte per element of FP8.

5.6 Additional measurement: speculative-decoding acceptance

The codec above is a deployable 4-bit model on its own terms, evaluated by perplexity in Tables 1 through 4. As one further measurement, not the paper's central claim, we evaluate it in the role of a speculative-decoding draft, where the target model verifies each proposed token, so the output distribution is that of the target and quantization loss reduces the acceptance rate rather than the output quality. This is a generic property of any accurate 4-bit model used this way; methods that

Table 5: Weight storage, bytes per element.

Component	bytes/element
E2M1 payload (4 bits)	0.500
Per-16 E4M3 block scale	0.0625
Per-tensor FP32 scale	≈ 0
FP8 low-rank factors, rank $d/6$	≈ 0.27
Total (W4A4 weights)	≈ 0.83
FP8 reference	1.000

train a draft head specifically for this task, such as Medusa and EAGLE (Section 8), are a different design point aimed at that task alone. Measured over 9,207 tokens against the FP16 target, the W4A4KV4 draft achieves an acceptance rate of $\alpha = 1 - \mathrm{TV} = 0.889$ and greedy agreement of 0.873. With draft block length $\gamma = 4$, the expected number of accepted draft tokens per verification step is $\sum_{k=1}^{\gamma} \alpha^k \approx 3.01$, and including the one token contributed by the target, the expected number of emitted tokens per step is $1 + \sum_{k=1}^{\gamma} \alpha^k \approx 4.01$. With a draft-to-target per-step cost ratio $c = 0.1$ the projected speedup over target-only decoding is $(1 + \sum_k \alpha^k)/(1 + \gamma c) = 4.01/1.4 \approx 2.86$. We report acceptance rather than wall-clock speedup because acceptance is independent of the inference kernel; a measured speculative throughput requires a deployed draft-and-target system and is left to future work, as is a larger evaluation with multiple domains, longer contexts, and confidence intervals.

5.7 Measured Blackwell throughput

Table 6 reports single-model decode throughput on a B200 using vLLM 0.23 and NVIDIA ModelOpt checkpoints, with input length 128 and output length 256.

Table 6: Decode throughput on a B200, tokens per second.

Batch	FP8	NVFP4	NVFP4/FP8
1	389.7	352.6	0.90
32	11,235	10,768	0.96
64	19,787	19,834	1.00

NVFP4 is slower than FP8 at single-stream decoding and reaches parity near batch size 64. Single-stream decoding is bounded by memory traffic and by the cost of unpacking the block scales, so the larger arithmetic throughput of FP4 is not exercised. The advantage of FP4 therefore depends on large models, mature kernels, and high concurrency. These numbers use NVIDIA's stock NVFP4 path rather than our corrected kernel.

6 Quantization-aware training beyond the post-training floor

The results above are a post-training codec: the weights are trained normally and quantization is applied afterward. Section 7 reports that this codec is close to a floor on every leg, and that training a low-rank correction of the residual after the fact does not improve on the analytical fit, because the residual is already close to white with respect to the weights. If the residual cannot be patched after training, the remaining lever is to train the weights themselves against the quantizer they will be deployed with, rather than patch them afterward. That lever, quantization-aware training

(QAT), is what this section is about: it keeps paying off where post-hoc correction cannot, and it gives a 4-bit model deployable on its own.

We insert the NVFP4 fake-quantizer into the forward pass with a straight-through gradient estimator and distill the quantized student against the FP16 teacher by KL divergence. Training must use the same quantizer as deployment: training against a proxy quantizer and exporting to a different one reintroduces a gap by construction.

TinyLlama-1.1B. With FP16 at 9.358 and FP8 at 9.388 (nearly lossless), raw post-training W4A4 is 10.734. Training for 3,000 steps against a fake quantizer that matches the deployment semantics of Section 2 reaches 9.768, closing 70% of the W4A4 gap. This reproduces end to end on a real Blackwell GPU (RTX Pro 6000, measured 9.766).

Removing the sim-to-deploy gap. Training against a proxy fake-quantizer and then exporting to a production quantization toolkit's own quantizer cost an additional 0.18 perplexity in our setting (deployed 9.94 versus the 9.77 measured during training), because the two quantizers differ in details such as calibration and rounding. Training natively inside the same quantizer used for export removes this gap by construction: the perplexity measured during training is the deployed perplexity. Doing so reaches a deployed 9.709, better than both the earlier deployed number and our own training-time estimate.

The KV cache is free under QAT, and three methods converge to a floor. Repeating native QAT with the KV cache also quantized to NVFP4 deploys at 9.705, statistically tied with the weights-and-activations-only result, so the KV-cache leg costs nothing further once QAT is used. Adding per-channel equalization on top of native QAT deploys at 9.707, no better than QAT alone. We read this as the two-lever theorem of Section 3 confirming itself in a different setting: once the model is trained to be robust to the quantizer, equalization has nothing left to correct. These three independently motivated methods converge to the same value, about 9.71, a gap of 0.32 over FP8. It is this convergence, more than any single number, that makes us treat 9.71 as a floor for this model and configuration rather than an artifact of one method.

Why the floor is 0.32 above FP8. FP8 is itself nearly lossless relative to FP16 (9.388 versus 9.358), so matching it requires 4-bit inference that is close to lossless as well. QAT makes the weights robust to quantization, but the activations and KV cache are still quantized at every forward pass at inference time, and that runtime quantization noise is not something training the weights alone can remove. Removing it would mean spending back one of the axes currently held at FP8 parity, for instance by keeping activations or the KV cache at higher precision, which departs from an all-4-bit configuration. This floor characterizes the current configuration; a different one might move it.

Scope. The results in this section are demonstrated at 1.1B-parameter scale. We have separately observed that the same recipe applied without modification at 8B parameters is harder: full-weight QAT does not fit alongside the FP16 teacher in consumer-class GPU memory, and a parameter-efficient variant we tried closes a smaller fraction of the gap under a limited training budget. The 1.1B result establishes the mechanism; the 8B result shows that scaling the exact recipe is not yet solved. We do not include the 8B numbers here because we have not reproduced them with the same train-equals-deploy property established in this section, and reporting a number that mixes the two would be misleading.

7 Limitations

The full 4-bit configuration is 0.42 perplexity above FP8. The activation codec appears near the limit of the lightweight post-training variants we tested, since the post-equalization residual is close to white and we did not improve it with several families of post-training methods; scale-optimization methods we do not evaluate [2, 5, 8] may move this limit.

Trained correction. We attempted to reduce the gap by distilling the weight low-rank factors against the teacher logits rather than fitting them by singular value decomposition, with a keep-best procedure that prevents regression below the SVD baseline. On Llama-3.1-8B this gave no improvement over the SVD factors. The reason is that the weight quantization is already near-lossless with respect to the teacher (per-token distillation loss of about 0.013 nats at the initialization), so the remaining W4A4KV4 gap comes from the activation and KV quantization rather than the weights. We separately attempted trained correction of the activation path directly; it gave a real but small gain that shrank at longer context (from about 0.10 to 0.14 down to 0.032 perplexity at 2048 tokens) and the training was fragile, since the activation residual is close to white for the same reason the weight factors are. Both null and fragile results point the same way: patching a residual after training has little room left to work with, which is what motivates training the weights directly against the quantizer instead (Section 6).

Baselines. Table 3 compares our codec against raw NVFP4 and a QuaRot-style Hadamard rotation under identical semantics and byte budget. We have not yet compared against a learned-rotation method [12], an external ModelOpt NVFP4 checkpoint, or NVFP4 scale-optimization methods [2, 5, 8] on the same eval. Until those are run, the comparisons reported above are what we can defend, not a best-in-class result.

Compute cost. The activation side-channel is inexpensive, but the weight low-rank correction at rank $d/6$ adds approximately $r/m + r/d$ of the main matrix multiply in compute-bound regimes, which is on the order of one third for square layers, and adds the corresponding factor traffic in memory-bound single-stream decoding. Lower ranks trade accuracy for cost. We did not measure deployed latency.

Deployment. The corrections do not have a production kernel, so deployment requires a fused FP4 epilogue in an inference engine such as TensorRT-LLM, and is worthwhile only if the epilogue cost stays below the accuracy benefit.

8 Related work

Rotation methods [1, 12] remove outliers by spreading activation energy across coordinates; our analysis predicts a small effect after equalization in our setting, and we treat their gains as a regime question rather than a contradiction. Weight-scaling heuristics such as AWQ [11] and error-feedback rounding such as GPTQ [7] act at the granularity the block scale already covers; we use GPTQ as a component. Low-rank reconstruction [14] reconstructs quantization residuals with an additive term, which is the lever we share and extend with a per-block sign-projection. Recent scale-selection methods such as SOAR [2], which jointly optimizes the global and block-wise scales, Four Over Six [5], which adaptively scales blocks to reduce near-maximal-value error, and ScaleSearch [8], which searches block floating-point scale candidates, report gains from adjusting block and tensor

scales, which motivates the baseline comparison we leave to future work. Lattice and codebook methods [4, 6] improve the representation but depart from the hardware-native execution path. Quantization-aware training for low-bit inference is an established practice, most visibly in Google's Gemma QAT checkpoints. What Section 6 contributes is less QAT itself and more the diagnosis of when it is needed, once post-hoc correction hits the floor identified in Sections 3 and 7, and the demonstration that training against the deployment quantizer removes the sim-to-deploy gap by construction. For the additional speculative-decoding measurement in Section 5 we use the two-model formulation [9]; Medusa [3] and EAGLE [10] instead train a feature-level draft head specifically for this task and are the stronger choice when the only goal is drafting for a fixed target. At that narrower task our model is not trying to compete with them: it is a general-purpose 4-bit model, evaluated on its own accuracy first, for which draft acceptance is one additional measurement rather than the design objective.

9 Conclusion

We have argued that under a high-resolution white-error model of the microscaling quantizer, two operations, equalization and additive low-rank correction, capture the first-order levers for NVFP4 accuracy, and that in a post-training setting several multiplicative and rounding refinements add little after equalization. This post-training codec is deployable on its own and is close to a floor set by a residual that is already close to white. Training the model directly against the deployment quantizer (QAT) is the lever that continues to pay off past that floor: it closes 70% of the remaining W4A4 gap on TinyLlama-1.1B, reproduces with no sim-to-deploy gap on real Blackwell hardware, and, together with a 4-bit KV cache and equalization, converges to a characterized all-4-bit floor about 0.32 perplexity above FP8 that we argue is fundamental to holding all-4-bit at once rather than an artifact of our specific methods. The outcome at each scale we tested is a 4-bit model that stands on its own; as one further measurement we evaluate the 8B post-training codec as a speculative-decoding draft and find 88.9% acceptance against the FP16 target. Because measured throughput shows no single-stream speed advantage for NVFP4 at this scale, we suggest that 4-bit methods of this kind be compared on accuracy per byte and memory footprint rather than single-stream speed, and we identify the equal-byte baseline study and scaling the train-equals-deploy QAT recipe past 1.1B parameters as the work needed to turn these measurements into a defensible comparison.

References

[1] S. Ashkboos, A. Mohtashami, M. L. Croci, et al. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. In *NeurIPS*, 2024.

[2] C. Bao, X. Yan, Z. Li, G. Qin, G. Yu, and Y. Zhang. SOAR: Scale Optimization for Accurate Reconstruction in NVFP4 Quantization. *arXiv preprint arXiv:2605.12245*, 2026.

[3] T. Cai, Y. Li, Z. Geng, et al. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. *arXiv preprint arXiv:2401.10774*, 2024.

[4] J. Chee, Y. Cai, V. Kuleshov, and C. De Sa. QuIP: 2-Bit Quantization of Large Language Models with Guarantees. In *NeurIPS*, 2023.

[5] J. Cook, J. Guo, G. Xiao, Y. Lin, K. Wyss, M. Nazemi, A. Mishra, C. del Mundo, T. Blankevoort, and S. Han. Four Over Six: More Accurate NVFP4 Quantization with Adaptive Block Scaling. *arXiv preprint arXiv:2512.02010*, 2025.

[6] V. Egiazarian, A. Panferov, D. Kuznedelev, et al. Extreme Compression of Large Language Models via Additive Quantization (AQLM). In *ICML*, 2024.

[7] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[8] T. Gupta, H. Prairie, X. Wu, R. Abhyankar, Q. Wu, A. Silveria, P. Ponnusamy, J. Wang, B. Athiwaratkun, L. Song, T. Dao, D. Y. Fu, and C. De Sa. Search Your Block Floating Point Scales! *arXiv preprint arXiv:2605.12464*, 2026.

[9] Y. Leviathan, M. Kalman, and Y. Matias. Fast Inference from Transformers via Speculative Decoding. In *ICML*, 2023.

[10] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

[11] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *MLSys*, 2024.

[12] Z. Liu, C. Zhao, I. Fedorov, et al. SpinQuant: LLM Quantization with Learned Rotations. In *ICLR*, 2025.

[13] NVIDIA. NVFP4: format and scaling in the Transformer Engine. NVIDIA Transformer Engine documentation, 2025.

[14] C. Zhang, J. Cheng, G. A. Constantinides, and Y. Zhao. LQER: Low-Rank Quantization Error Reconstruction for LLMs. In *ICML*, 2024.